

-Project Report- ML-Powered Gesture-Controlled Retro Boxing Game ([Github](#))

Developed by: [Nikhil Yadav](#)

1. Project Overview

This project is a browser-based, retro-style arcade boxing game that replaces traditional keyboard or controller inputs with real-time human body tracking. By utilizing a webcam and machine learning computer vision, players control their on-screen character using physical movements—leaning to move and thrusting their wrists to punch.

Github Link - <https://github.com/ItsYadavNikhil/Boxing-Game>

2. Technology Stack

- **Frontend Rendering:** HTML5 `<canvas>` API (for high-performance 2D drawing)
 - **Game Logic:** Vanilla JavaScript (ES6+)
 - **Machine Learning / Computer Vision:** Google MediaPipe Pose (for real-time skeletal tracking)
 - **Audio Engine:** Web Audio API (for low-latency, overlapping 8-bit sound effects)
 - **Asset Pipeline:** Free Texture Packer (for optimizing sprite sheets)
-

3. System Architecture & Data Flow

The game operates on a continuous, high-speed loop (60 Frames Per Second), with two distinct systems running in parallel: the **AI Vision System** and the **Game Engine**.

Phase 1: Input & AI Processing (MediaPipe Pose)

1. **Webcam Capture:** The browser requests access to the user's webcam and feeds a continuous video stream to the ML model.
2. **Landmark Extraction:** MediaPipe Pose analyzes each frame and identifies 33 3D landmarks on the human body.
3. **Gesture Translation:**
 - **Movement (The Nose Lean):** The system tracks the X-coordinate of Landmark 0 (the Nose). If the nose crosses a defined threshold to the left or right of the screen's center line, it triggers the left/right movement variables.
 - **Combat (The Wrist Thrust):** The system tracks Landmarks 15 and 16 (Left and Right Wrists). Sudden velocity or depth (Z-index) changes trigger the punch variables.
4. **Virtual Keystrokes:** The ML logic maps these physical gestures to the game's internal `keys` object, artificially "pressing" the arrows and action buttons.

Phase 2: The Game Engine (Canvas & JavaScript)

1. **State Machine:** The engine checks the input variables and updates the state of the Player and Enemy (IDLE, WALK, PUNCH, BLOCKING, HURT).
 2. **Physics & Collision Detection:**
 - The game calculates positions and checks for overlaps between the characters' custom "hitboxes" (invisible mathematical rectangles).
 - It enforces boundaries ("Ring Ropes") to keep characters on screen.
 3. **Audio Triggering:** If a state changes (e.g., a punch is thrown) or a collision occurs (e.g., a punch lands on a hitbox), the Web Audio API clones and plays the corresponding .wav file (miss.wav, punch.wav, blocked.wav, etc.).
 4. **Sprite Rendering:** Based on the current state and a frame counter, the engine calculates the `sourceX` and `sourceY` coordinates to cut the correct animation frame out of the master sprite sheet. It applies horizontal flipping (mirroring) if characters switch sides, scales the pixel art up by 3x, and draws it to the canvas.
-

4. Key Technical Implementations

Decoupled Hitboxes (Fixing the "Phantom Hit")

A major challenge in 2D fighting games is that the visible pixel art rarely matches the strict rectangular boundaries of the image file. To prevent characters from colliding with the invisible "air" around the sprites, the collision math was decoupled from the rendering engine. We implemented:

- **Custom Body Hitboxes:** Snug, mathematical boundaries wrapped strictly around the character's core.
- **Custom Punch Reach:** A dynamic line of collision that only extends outward during specific animation frames.
- *Note:* A developer toggle (bound to the '0' key) was included to overlay these hitboxes visually for precise tuning.

Dynamic Aspect Ratio Management

To ensure the boxing ring background (bg.png) filled the screen without squishing or distorting the pixel art, CSS-style `object-fit: cover` logic was applied natively in the Canvas draw loop, preserving the aspect ratio while bleeding excess edges off-screen.

Audio Overlap & Autoplay Compliance

Modern browsers block autoplaying audio. The game handles this by requiring a "Click to Start" screen, which simultaneously starts the game loop, plays the `Openingbell.wav` to unlock the Audio Context, and triggers the looping background crowd noise (`bgnoise.wav`). Furthermore, all audio files use `.cloneNode(true)` before playing to ensure rapid, successive punches do not cut off the previous sound effect.

5. Future Enhancements

- **Multiplayer Capability:** Utilizing WebRTC to send MediaPipe gesture data over a peer-to-peer connection for remote PvP fights.
- **Stamina System:** Tracking the physical speed and frequency of the user's punches to introduce a digital fatigue meter, encouraging strategic pacing rather than chaotic flailing.